Lesson 3

Modifying Data

HUMBER
School of Media Studies
& Information Technology

INSERT, UPDATE and DELETE Statements

- In the SQL programming language, data manipulation language (DML) is designed to add, remove and modify data
- The INSERT, UPDATE and DELETE commands provide you with the ability to manipulate data

HUMBER
School of Media Studies
& Information Technology

- The table schema (table definition) affects the values that may or may not be defined, modified and removed in INSERT, UPDATE and DELETE statements
  - IDENTITY property
  - Allow Nulls property
  - TimeStamp data type

HUMBER
School of Media Studies
& Information Technology

- **Important related functions:**
  - @@IDENTITY: Returns last generated Identity by SQL Server for current session
  - SCOPE_IDENTITY: Returns the last IDENTITY value generated by SQL Server limited to the current session and scope only (discards IDENTITY generated by Triggers)
  - IDENT_CURRENT: Returns the value of the last IDENTITY generated for a table. It requires a table name parameter

HUMBER
School of Media Studies
& Information Technology

- You cannot enter a value for an identity column when executing an INSERT or UPDATE statement

- To overcome this limitation in current connection:

```
SET IDENTITY_INSERT <Table Name> ON
```

- NULL definitions and constraints also have an effect on DML statements

HUMBER
School of Media Studies
& Information Technology

- DML statements always succeed or fail as a whole
  - If you were inserting N records into a table, but one violated a PRIMARY KEY or UNIQUE constraint, all N rows would roll back and nothing would be inserted

HUMBER
School of Media Studies
& Information Technology

Inserting Data

- The INSERT INTO statement provides you with the ability to add new rows to a table
- Depending on the definition of the table, you may need to provide data for all or only a portion of the columns in the table
- DEFAULT constraints, IDENTITY properties and NULL settings may all affect the data requirements of your INSERT statements

HUMBER
School of Media Studies
& Information Technology

INSERT…VALUES: Insert a list of data values into an existing table

INSERT…SELECT: Insert the result of an inline query into an existing table

INSERT…EXECUTE: Insert the results of a stored procedure into a new table

INSERT TOP: Insert a specified number, or percent of rows from a query

SELECT…INTO: Create a new table based on query results

HUMBER
School of Media Studies
& Information Technology

```
INSERT [INTO] table_name
 [(comma separated list of column names)]
VALUES ({data_value | DEFAULT | NULL | Scalar
Expression}, …n), […n]
```

- You can not insert values into computed columns
- Best practice is to specify column names, as this helps avoid issues in the future when the column structure changes

HUMBER
School of Media Studies
& Information Technology

Inserting 3 record

```
INSERT HumanResources.Department
   (GroupName, Name, ModifiedDate)
VALUES
('Executive General and Administration',
'Database Administration',
GETDATE()),

('Inventory Management', 'Stock Control',
GETDATE()),

('Executive General and Administration', 'Legal',
GETDATE());
```

HUMBER
School of Media Studies
& Information Technology

Using the *INSERT Statement*

```
CREATE TABLE [Sales].[SalesReason]
(
        [SalesReasonID] [int] IDENTITY(1,1) NOT NULL,
        [Name] [dbo].[Name] NOT NULL,
        [ReasonType] [dbo].[Name] NOT NULL,
        [ModifiedDate] [datetime] NOT NULL,
CONSTRAINT [PK_SalesReason_SalesReasonID]
PRIMARY KEY CLUSTERED ([SalesReasonID] ASC);
GO

INSERT INTO Sales.SalesReason
VALUES ('Item Closeout', 'Other', getdate());
GO
```

- In this command, SalesReasonID is not entered because it is an identity column and is inserted automatically by the SQL Server engine

HUMBER
School of Media Studies
& Information Technology

```sql
ALTER TABLE [Sales].[SalesReason]
ADD CONSTRAINT [DF_SalesReason_ModifiedDate]
DEFAULT (getdate()) FOR [ModifiedDate]
GO
```

- The ModifiedDate column of the SalesReason table has a DEFAULT constraint associated with it that specifies the getdate() function
- It is the same

```sql
INSERT INTO Sales.SalesReason
VALUES ('Item Closeout', 'Other', DEFAULT );
```

Note: The keyword “Into” is optional

HUMBER
School of Media Studies
& Information Technology

- If a column supports NULL values, you can use the keyword NULL to insert a row with missing data in one or more columns

```
INSERT INTO Sales.SpecialOffer
VALUES ('Temporary Holiday
Discount', .10, 'Seasonal Discount',
'Reseller', getdate(), DATEADD (dd, 30,
getdate())
    , 1, NULL, DEFAULT, DEFAULT);
```

HUMBER
School of Media Studies
& Information Technology

- To specify only certain columns, reference the column names when writing the INSERT statement:

```
INSERT INTO Sales.SpecialOffer
(Description, DiscountPct, Type, Category,
StartDate, EndDate, MinQty)

VALUES ('Temporary Holiday Discount', 0.10,
'Seasonal Discount', 'Reseller', GETDATE(),
DATEADD (dd, 30, GETDATE()), 1);
```

HUMBER
School of Media Studies
& Information Technology

- INSERT . . . SELECT statement is used to append rows to an existing table based on data selected from a different table

```
INSERT [INTO] table_or_view

[(comma separated list of column names)]

derived_table
```

SELECT statement with same number of columns and data types

HUMBER
School of Media Studies
& Information Technology

```sql
use AdventureWorks2008;
GO

CREATE TABLE dbo.Colors
(ColorID int IDENTITY NOT NULL PRIMARY KEY,
Color nvarchar(15));
GO

INSERT Colors
SELECT DISTINCT Color
FROM  Production.Product
WHERE Color is not NULL
ORDER BY Color;
GO
```

HUMBER
School of Media Studies
& Information Technology

The following command takes rows from the Employees table and appends them to the Customers table

```
INSERT INTO Sales.Customers
SELECT
        'Northwind Traders',
        FirstName + ' ' + LastName,
        'Employee',
        Address,
        City,
        Region,
        PostalCode,
        Country,
        Phone,
        NULL
FROM HR.Employees;
```

HUMBER
School of Media Studies
& Information Technology

- The INSERT….EXECUTE keyword allows you to execute a stored procedure or to interpret a string as a Transact-SQL command and execute the resultant command

HUMBER
School of Media Studies
& Information Technology

```sql
CREATE PROC GetMostPopularColors
AS
SELECT Color
FROM Production.Product
JOIN Sales.SalesOrderDetail
ON Production.Product.ProductID =
Sales.SalesOrderDetail.ProductID
WHERE Color is not null
GROUP BY Color Having COUNT(*) > 10000;
GO

INSERT Colors (Color)
EXECUTE GetMostPopularColors;
GO
```

Create SP

Use SP to INSERT

HUMBER
School of Media Studies
& Information Technology

- SELECT INTO statement allows you to create a new temporary or permanent table populated with the results of the defined SELECT statement

- The statement copies from the source
  - Column names
  - Column Data Types
  - Null ability
  - IDENTITY property
  - Actual Data

HUMBER
School of Media Studies
& Information Technology

```
SELECT *
INTO Person.Person_Backup
FROM Person.Person;
```

```
SELECT *
INTO Person.Person_Backup IN 'a_DB_name'
FROM Person.Person;
```

```
SELECT LastName,FirstName
INTO Person.Person_Backup
FROM Person.Person
Where Title ='Mr.';
```

HUMBER
School of Media Studies
& Information Technology

- If you want target columns to be defined different than the source, you need to apply some manipulatio

Get rid of IDENTITY property

Make column NOT NULL

```
SELECT
  ISNULL(orderid + 0, -1) AS orderid,
            -
  ISNULL(custid, -1) AS custid,
  empid,
  ISNULL(CAST(orderdate AS DATE),'19000101') AS orderdate
  , shipcountry , freight

INTO Sales.MyOrders
FROM Sales.Orders
WHERE shipcountry = N'Norway';
```

HUMBER
School of Media Studies
& Information Technology

- INSERT ....TOP
- The TOP (n) or TOP (n) PERCENT clause specifies that only the first n number or percent of rows returned from the query result should be inserted using the INSERT statement.

```
INSERT table_name
TOP (n) [PERCENT]
[WITH TIES]
SELECT … FROM … [JOIN…][WHERE…]
```

HUMBER
School of Media Studies
& Information Technology

```
CREATE TABLE dbo.SickLeave
(     EmployeeID int,
      FullName nvarchar (100),
SickLeaveHours int
);
GO

INSERT TOP (10) PERCENT SickLeave
SELECT EmployeeID,
FirstName + ' ' +  LastName,
SickLeaveHours
FROM Person.Contact JOIN
  HumanResources.Employee
ON Person.Contact.ContactID =
  HumanResources.Employee.ContactID
ORDER BY SickLeaveHours DESC;
```

HUMBER
School of Media Studies
& Information Technology

Updating Data

- The UPDATE command allows you to change the value of one or more columns in one or more rows of a table

- Adding a value to a column that is currently NULL and removing a value from a column are both considered UPDATE statements, not INSERT or DELETE statements

HUMBER
School of Media Studies
& Information Technology

```
UPDATE
    [ TOP (n) [PERCENT] ]
table_or_view
SET
{ column_to_update = { new_value | DEFAULT | NULL }
| column_to_update { .WRITE (new_value, @Offset,@Length )}
| @variable = new_value
| @variable = column_to_update = new_value
| column_to_update {+= | -= | *= | /= | %= | &= | ^= | |= } new_value
| @variable { += | -= | *= | /= | %= | &= | ^= | |= } new_value
| @variable = column_to_update
{ += | -= | *= | /= | %= | &= | ^= | |= } new_value }
[FROM …
WHERE … ]
```

- In its simplest form, the UPDATE statement requires only the name of the object (table or view) being updated and the value to be set

```
UPDATE Sales.CurrencyRate
SET EndOfDayRate = 2.805;
```

- The WHERE clause defines which rows are updated by the command

HUMBER
School of Media Studies
& Information Technology

- UPDATE statement has the ability to update all rows in a table with a single command

- The following command adds 0.005 to EndOfDayRate column in every row in the CurrencyRate table

```
UPDATE Sales.CurrencyRate
SET EndOfDayRate = EndOfDayRate + .005;
```

HUMBER
School of Media Studies
& Information Technology

- Using UPDATE with WHERE clause:

```
UPDATE Sales.SalesReason
SET Name = 'N/A'
WHERE SalesReasonID = 10;
```

HUMBER
School of Media Studies
& Information Technology

- You can update the information in a target table based on information in a table joined to the target table by using the FROM clause
- To increases the unit price of all red products by 5 percent:

```
UPDATE Sales.SalesOrderDetail
SET UnitPrice = UnitPrice * 1.05
FROM SALES.SalesOrderDetail
JOIN Production.Product
ON SALES.SalesOrderDetail.ProductID =
Production.Product.ProductID
WHERE Production.Product.Color = 'Red';
```

HUMBER
School of Media Studies
& Information Technology

- Update and Table Expressions

```
With C as
(
 SELECT *
 FROM SALES.SalesOrderDetail
 JOIN Production.Product
 ON SALES.SalesOrderDetail.ProductID =
 Production.Product.ProductID
 WHERE Production.Product.Color = 'Red'
)
UPDATE C
SET UnitPrice = UnitPrice * 1.05;
```

HUMBER
School of Media Studies
& Information Technology

Deleting Data

```
DELETE
    [ TOP ( expression ) [ PERCENT ] ]
    [ FROM ]
    { <object> | rowset_function_limited
      [ WITH ( <table_hint_limited> [ ...n ] ) ]
    }
    [ <OUTPUT Clause> ]
    [ FROM <table_source> [ ,...n ] ]
    [ WHERE { <search_condition>
            | { [ CURRENT OF
                    { { [ GLOBAL ] cursor_name }
                        | cursor_variable_name
                    }
                ]
              }
            }
    ]
    [ OPTION ( <Query Hint> [ ,...n ] ) ]
[; ]
```

HUMBER
School of Media Studies
& Information Technology

Using the DELETE Statement

- Like the UPDATE statement, it is important to remember to include the WHERE clause when writing a DELETE statement

- If you execute a DELETE statement without a WHERE clause, **all** rows from the table are deleted

HUMBER
School of Media Studies
& Information Technology

- To delete all rows from a table, the syntax is simply
- FROM keyword is optional

```
DELETE FROM <tablename>
```

HUMBER
School of Media Studies
& Information Technology

- To limit the rows removed, add a WHERE clause to the DELETE statement

```
DELETE FROM EMPLOYEES
WHERE EmployeeID = 7;
```

HUMBER
School of Media Studies
& Information Technology

- The DELETE command can remove rows in one table based on information returned from a joined table
- To remove rows from the Order Details table where the order was placed before July 10, 1996 and the order has shipped

```
DELETE FROM [Order Details]
FROM ORDERS
JOIN [Order Details]
ON
Orders.OrderID = [Order Details].OrderID
WHERE OrderDate < '07-10-1996'
AND ShippedDate IS NOT NULL;
```

HUMBER
School of Media Studies
& Information Technology

- DELETE using CTE

```
With C AS
(
SELECT *
FROM ORDERS
JOIN [Order Details]
ON Orders.OrderID = [Order Details].OrderID
WHERE OrderDate < '07-10-1996' AND ShippedDate IS
NOT NULL
)
DELETE FROM C;
```

HUMBER
School of Media Studies
& Information Technology

TRUNCATE TABLE

- The TRUNCATE TABLE statement can be used to remove all data from a table

- The TRUNCATE TABLE statement differs from the DELETE statement in several ways

HUMBER
School of Media Studies
& Information Technology

1. The DELETE statement logs information on each row deleted, while the TRUNCATE TABLE statement only creates entries for the deallocation of the data pages
2. Because of the minimal logging of the data from the table, the TRUNCATE TABLE statement executes faster and requires fewer resources on the server
3. If an identity column exists in the table, the TRUNCATE TABLE command resets the identity seed value

HUMBER
School of Media Studies
& Information Technology

- The syntax for the TRUNCATE TABLE statement is simply

```
TRUNCATE TABLE <tablename>
```

HUMBER
School of Media Studies
& Information Technology